**TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN - ĐHQG TP.HCM**
**KHOA CÔNG NGHỆ THÔNG TIN**


NHẬP MÔN TRÍ TUỆ NHÂN TẠO
BÁO CÁO LAB 01

# VIETNAMESE TRAFFIC ROUTE OPTIMIZATION

Search Algorithms on a Ho Chi Minh City Road Graph

**Giảng viên:** Bùi Tiến Lên
**Giảng viên:** Bùi Duy Đăng
**Giảng viên:** Võ Nhật Tân

———-oOo———-

**Sinh viên thực hiện**

| | |
|---|---|
| Dương Trung Hiếu | 24127040 |
| Nguyễn Hoàng Bảo Long | 24127201 |
| Trần Vũ Anh Kiệt | 24127196 |

Thành phố Hồ Chí Minh, tháng 8 năm 2026

# Abstract

This technical report describes a traffic route optimization application for a directed road graph in Ho Chi Minh City. The dataset contains 212 geographic vertices and 1,129 directed adjacency entries after two-way roads are expanded. Road segments store distance, nominal travel time, congestion, direction, road type, and risk factors. The project implements DFS, BFS, UCS, Dijkstra, A*, and Greedy Best-First Search for two-location routing, together with prototype Nearest-Neighbor and Held-Karp methods for multi-location routing. A FastAPI backend exposes graph and search operations, while a React and Leaflet interface displays the network and route controls.

Experiments on a reachable 28-hop test case show that UCS, Dijkstra, and A* produce the same minimum distance-mode cost. A* expands 158 vertices instead of 198 for UCS, while Greedy Best-First expands only 34 but returns a route whose cost is 6.1 percent higher. In a five-location cycle, Held-Karp reduces mixed cost by 11.32 percent relative to the supplied visit order and by 26.04 percent relative to Nearest Neighbor. The report also records integration gaps found in the current repository so that completion claims remain reproducible.

**Keywords:** graph search, traffic routing, A*, Dijkstra, multi-location optimization, FastAPI, React, Ho Chi Minh City.

# Document Conformance

This report is organized directly from Section 4.9, “Technical Report,” of the Lab 1 specification. Chapters 1 through 10 correspond to items (a) through (j): group introduction, problem context, modeling, dataset, algorithm principles, program flow, comparison, multi-location optimization, program instructions, and limitations/future work. The assessment in Chapter 1 is based on the repository state inspected on August 15, 2026.

# Contents

# List of Figures

# List of Tables

CHAPTER 1

# Group Introduction

## 1.1 Group Information

The project is prepared by Group 314 for Lab 1 of Introduction to Artificial Intelligence. Table 1.1 records the submitted member list.

Table 1.1: Group members

| No. | Student | Student ID |
|---|---|---|
| 1 | Dương Trung Hiếu | 24127040 |
| 2 | Nguyễn Hoàng Bảo Long | 24127201 |
| 3 | Trần Vũ Anh Kiệt | 24127196 |

## 1.2 Member Contributions

Work was divided by technical area, with joint review of integration and the final demonstration.

Table 1.2: Specific contribution of each member

| Member | Main contribution |
|---|---|
| Nguyễn Hoàng Bảo Long (Data & Model) | Curated the road dataset, defined and validated the cost model, documented the data and modeling, compared algorithms, and finalized the report and demo materials. |
| Trần Vũ Anh Kiệt (Core Algorithms) | Designed the graph structure; implemented BFS, DFS, UCS, A*, advanced and multi-location search; and optimized and debugged the algorithms. |
| Dương Trung Hiếu (Web/Local GUI) | Built the interface, map, route selection, traversal animations, route explanations and metrics, and supported backend integration. |

## 1.3 Requirement Completion Level

Table 1.3 distinguishes implemented functions from prototypes and integration work still required. This avoids treating source-code presence as proof of end-to-end completion.

Table 1.3: Completion status against the Lab 1 requirements

| Requirement | Status | Evidence / remaining work |
|---|---|---|
| Vietnamese traffic graph and attributes | **Completed** | 212 nodes; distance, time, congestion, type, direction, and risks. |
| Two-location search | **Completed** | DFS, BFS, UCS, Dijkstra, A*, and Greedy return a common result model. |
| Traffic-aware cost and heuristics | **Completed** | Distance, time, and mixed modes plus Haversine heuristics. |
| Backend API | **Completed** | Graph information and route-search endpoints are implemented. |
| GUI and graph display | **Prototype / partial** | Map and controls render; current search URL/label mapping needs alignment with the API. |
| Step-by-step visualization | **Prototype / partial** | Playback controls exist, but the displayed step counter is not yet driven by the returned exploration list. |
| Multi-location optimization | **Prototype / partial** | Nearest Neighbor and Held-Karp exist; an obsolete A* keyword prevents direct execution without a compatibility wrapper. |
| Route explanation | **Prototype / partial** | An explanation card exists, but its text is generic rather than generated from route segments. |
| Automated testing | **Prototype / partial** | Smoke tests run; the legacy runner uses an obsolete cost API and the current default pair can be unreachable. |

CHAPTER 2

# Problem Context

## 2.1 Selected Vietnamese Traffic Scenario

The selected scenario is an educational urban navigation system for a subset of Ho Chi Minh City. A user chooses a start intersection, a destination, optional stops, a search algorithm, and an optimization objective. The system then searches a directed street graph and visualizes the route.

Ho Chi Minh City traffic is an appropriate context because one-way roads, variable congestion, narrow streets, construction, and flooding can make a short route slower or less suitable than an alternative. Therefore, route quality cannot be represented by physical distance alone.

## 2.2 Real-World Problem

For a two-location query, the system must return a valid directed path whose objective cost is minimized or approximated by the chosen algorithm. For a multi-location query, it must decide both the visiting order and the detailed road path between consecutive stops. Users also need an explanation: whether the route is shortest by distance, fastest by adjusted time, or best under the combined cost.

## 2.3 Why Optimization Is Useful

Traffic-aware routing can reduce travel time, avoid risky segments, and make algorithm behavior understandable. The project is also an instructional tool: the explored-node sequence shows why uninformed and heuristic searches behave differently on the same road network.

CHAPTER 3

# Problem Modeling

## 3.1 State-Space Formulation

The road network is a directed weighted graph $G = (V, E)$:

- A **state** is the current vertex identifier.
- The **initial state** is the selected start vertex $s$.
- The **goal test** succeeds when the current identifier equals $t$.
- A **transition** follows an outgoing edge in the adjacency list.
- A one-way street creates one transition; a two-way street creates two.
- A **solution** is an ordered sequence from $s$ to $t$ that respects direction.

## 3.2 Nodes and Edges

A node contains `node_id`, `name`, latitude, longitude, and a type such as intersection. An edge contains source, target, physical distance, nominal time, congestion level 1–5, road type, direction, and a list of risks. The adjacency-list design uses $O(|V| + |E|)$ memory.

## 3.3 Traffic-Aware Cost Function

Let $d(e)$ be physical distance, $t(e)$ nominal time, and $c(e)$ congestion. The implementation defines

$$d_{\text{eff}}(e) = d(e)[1 + 0.1(c(e) - 1)], \tag{3.1}$$

$$t_{\text{eff}}(e) = t(e)m(c(e)) + r(e), \tag{3.2}$$

where $m(c)$ is $1.0, 1.3, 1.8, 2.4$, or $3.5$. The risk term adds 300 seconds for flooding, 120 for construction, 60 for a narrow road, and 45 for a complex intersection. The selected edge cost is

$$C(e) = \begin{cases} d_{\text{eff}}(e), & \text{distance mode,} \\ t_{\text{eff}}(e), & \text{time mode,} \\ d_{\text{eff}}(e) + 10t_{\text{eff}}(e), & \text{mixed mode.} \end{cases} \tag{3.3}$$

Congestion therefore changes both effective distance preference and travel time. Mixed

mode converts one second into 10 metres of equivalent cost, a design choice based on an assumed urban speed near 10 m/s. The value is useful for ranking routes within mixed mode; it is not a monetary cost and must not be compared numerically with the other modes.

## 3.4 Assumptions

All costs are non-negative. Traffic values are static during one search. GPS coordinates are accurate enough for straight-line heuristics, while edge times, congestion, and risks are simulated attributes rather than live measurements. The system assumes the selected vertices belong to a reachable directed region.

CHAPTER 4

# Dataset

## 4.1 Creation Method and Scale

The project uses hybrid data: real street/intersection names and GPS coordinates from a Ho Chi Minh City area, augmented with simulated traffic conditions. The source JSON contains 212 nodes and road-segment records. When two-way segments are expanded in memory, the adjacency lists contain 1,129 directed edges. This exceeds the minimum Lab 1 requirement of 20 nodes and 30 edges.

## 4.2 Representative Locations

The complete location list is stored in `data/nodes.json`. Representative vertices used in experiments are shown below.

Table 4.1: Representative locations from the dataset

| ID | Location | Role in experiments |
|---|---|---|
| 123 | Đường Nguyễn Văn Cừ | Two-location start |
| 151 | Ngã giao Đường Nguyễn Trãi - Đường Phú Giáo | Two-location goal |
| 125 | Ngã giao Đường Hải Thượng Lãn Ông - Đường Võ Văn Kiệt | Multi-location depot |
| 65 | Ngã giao Đường Tản Đà - Đường Trần Hưng Đạo | Required stop |
| 192 | Đường Trang Tử | Required stop |
| 155 | Ngã giao Vĩnh Viễn - Ngô Quyền | Required stop |
| 81 | Ngã giao Cách Mạng Tháng Tám - Bắc Hải | Required stop |

## 4.3 Attribute Semantics

Distance is stored in metres and nominal time in seconds. Congestion is an integer from 1 (smooth) to 5 (gridlock). Direction is `one-way` or `two-way`. Road type distinguishes local roads, avenues, and alleys. Risk factors include flooding, construction, narrow roads, and complex intersections.

## 4.4 Data Quality and Connectivity

The largest strongly connected component contains 200 of the 212 vertices. The remaining directed structure explains why arbitrary first/last JSON entries may produce a no-path result. Experiments therefore use explicitly verified pairs. The data is suitable for algorithm comparison but does not claim citywide map coverage or real-time accuracy.

CHAPTER 5

# Algorithm Principles

## 5.1 Common Search Representation

Each frontier entry is a `SearchNode` with a vertex ID, parent, accumulated cost $g$, heuristic $h$, priority $f$, and incoming edge. When the goal is found, parent pointers reconstruct the route. All algorithms return a `SearchResult` containing path, explored order, cost, distance, time, count, and execution time.

## 5.2 Uninformed Search

### 5.2.1 Depth-First Search

DFS uses a stack and expands the most recently generated state. On a small example $S \rightarrow \{A, B\}$, if $B$ is pushed last, DFS explores the branch through $B$ first. It is complete on this finite graph with visited-state checking, but it is not optimal by hops or cost and depends strongly on adjacency order.

### 5.2.2 Breadth-First Search

BFS uses a queue and expands all depth-$k$ states before depth $k + 1$. In the same example it compares both one-edge branches before deeper states. It is complete and optimal only for number of edges when each step has equal cost; it does not minimize weighted traffic cost.

### 5.2.3 Uniform-Cost Search

UCS uses a min-heap ordered by $g(n)$, so the cheapest partial route is expanded first. With non-negative edges it is complete (subject to a positive minimum step cost) and optimal. A higher-hop route may be selected if its accumulated traffic cost is lower.

## 5.3 Additional Single-Route Algorithms

### 5.3.1 Dijkstra's Algorithm

For a single target and non-negative edge costs, Dijkstra has the same expansion rule as UCS. The project intentionally exposes it as a separate algorithm but delegates to the UCS implementation. It is complete and optimal under the current cost model.

### 5.3.2 A* Search

A* combines known and estimated cost:

$$f(n) = g(n) + h(n). \tag{5.1}$$

It expands the lowest $f$. The heuristic is Haversine distance in distance mode, distance divided by 16.67 m/s in time mode, and

$$h_{mixed}(n) = d_H(n,t) + 10d_H(n,t)/16.67 \tag{5.2}$$

in mixed mode. Because straight-line travel and maximum-speed travel are lower bounds, the intended heuristic is admissible. With the same lower-bound metric on every transition it is also intended to be consistent. This preserves optimality while often reducing expansions relative to UCS.

### 5.3.3 Greedy Best-First Search

Greedy Best-First orders the heap only by $h(n)$. It moves geographically toward the goal and may explore very few vertices, but it ignores accumulated cost. Therefore it is not optimal; the visited set makes it terminate on the finite dataset, though a poor heuristic can produce a poor route.

## 5.4 Multi-Location Methods

Nearest Neighbor repeatedly chooses the cheapest unvisited destination and is fast but approximate. Held-Karp dynamic programming stores the best path for each subset/current-stop pair. It guarantees an optimal tour over the selected stops when pairwise route costs are correct, at $O(n^2 2^n)$ time and $O(n2^n)$ space.

CHAPTER 6

# Program Flow

## 6.1 System Architecture

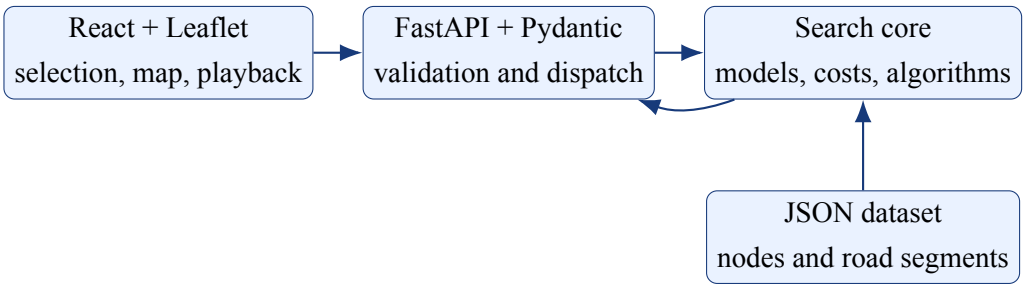


Figure 6.1: Three-layer system architecture

## 6.2 Main Processing Flow

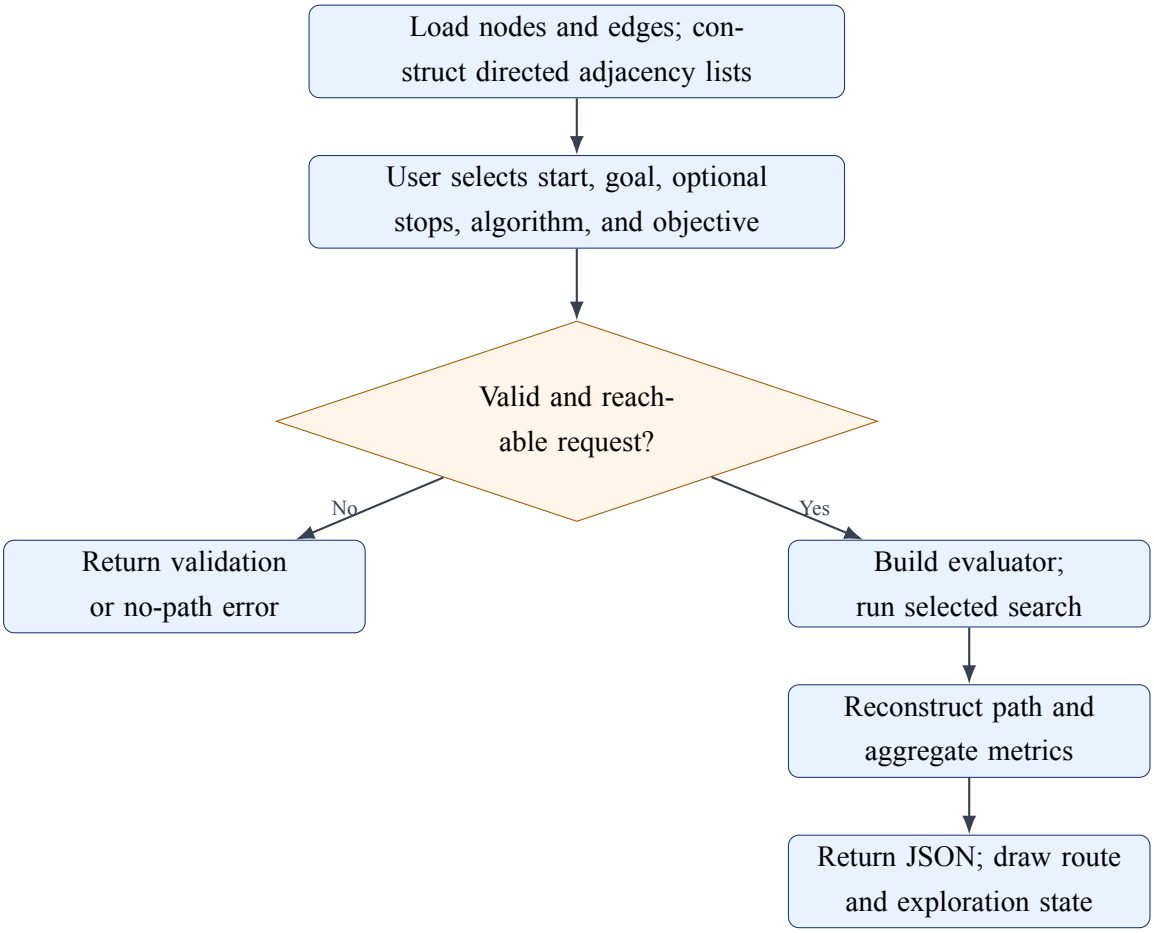


Figure 6.2: Main route-search flow

## 6.3 Modules and Responsibilities

Table 6.1: Program modules

| Module | Responsibility |
|---|---|
| `src/models.py` | Node/edge data classes, graph loader, risk and edge-cost calculation. |
| `src/heuristics.py` | Haversine distance and maximum-speed time estimate. |
| `src/algorithms/base.py` | Shared search-node/result types and path reconstruction. |
| `src/algorithms/uninformed.py` | DFS and BFS. |
| `src/algorithms/informed.py` | UCS, Dijkstra, A*, and Greedy Best-First. |
| `src/algorithms/multi_location.py` | Pairwise A* matrix, Nearest Neighbor, and Held-Karp. |
| `backend/api.py` | Graph endpoints, validation, algorithm dispatch, and response assembly. |
| `frontend/src/` | Input controls, map, settings, route results, and playback controls. |

## 6.4 GUI Interaction

The frontend imports graph data for immediate map display. A search request is intended to send start, end, algorithm, and optimization to FastAPI. The backend selects a `CostEvaluator`, dispatches the algorithm, and returns path IDs, coordinates, names, exploration order, and metrics. The current branch must align `/search` with `/api/search` and normalize labels such as `A*` to the backend value before this flow is fully end-to-end.

CHAPTER 7

# Algorithm Comparison

## 7.1 Theoretical Comparison

Table 7.1: Complexity, completeness, and optimality

| Algorithm | Time | Memory | Complete | Optimal |
|---|---|---|---|---|
| DFS | $O(V+E)$ | $O(V)$ | Yes, finite graph | No |
| BFS | $O(V+E)$ | $O(V)$ | Yes | By hop count only |
| UCS | $O((V+E)\log V)$ | $O(V)$ | Yes | Yes, non-negative costs |
| Dijkstra | $O((V+E)\log V)$ | $O(V)$ | Yes | Yes, non-negative costs |
| A* | Exponential worst case | Exponential worst case | Yes under standard conditions | Yes with admissible/consistent $h$ |
| Greedy Best-First | Exponential worst case | Exponential worst case | Yes on this finite implementation | No |
| Nearest Neighbor | $O(n^2)$ after cost matrix | $O(n)$ | Produces a tour if connected | No |
| Held-Karp | $O(n^2 2^n)$ | $O(n2^n)$ | Yes | Yes for selected stops |

## 7.2 Actual Two-Location Performance

The verified query is vertex 123 (Đường Nguyễn Văn Cừ) to vertex 151 (Ngã giao Đường Nguyễn Trãi - Đường Phú Giáo) in distance mode. One local Python 3.13 run produced Table 7.2. Sub-millisecond differences are illustrative; route quality and expansions are more meaningful on this small graph.

Table 7.2: Actual distance-mode results from vertex 123 to 151

| Method | Nodes | Dist. (m) | Time (s) | Cost | Expanded | ms |
|---|---|---|---|---|---|---|
| DFS | 87 | 16,783.30 | 19,499.50 | 20,546.06 | 134 | 0.594 |
| BFS | 28 | 4,943.32 | 4,927.50 | 5,878.60 | 204 | 0.299 |
| UCS | 31 | 4,781.20 | 3,648.00 | 5,442.72 | 198 | 1.108 |
| Dijkstra | 31 | 4,781.20 | 3,648.00 | 5,442.72 | 198 | 1.081 |
| A* | 31 | 4,781.20 | 3,648.00 | 5,442.72 | 158 | 1.162 |
| Greedy Best-First | 32 | 4,855.48 | 4,810.00 | 5,775.63 | 34 | 0.146 |

UCS, Dijkstra, and A* agree on the optimum. A* reduces expansions by 20.2 percent relative to UCS. Greedy expands only 34 vertices but costs 6.1 percent more than the optimum. BFS finds the fewest-edge route, not the cheapest route, and DFS follows a long adjacency-order-dependent detour.

## 7.3 Effect of Traffic Objective

Table 7.3: A* results under three optimization criteria

| Mode | Nodes | Dist. (m) | Time (s) | Cost | Expanded |
|---|---|---|---|---|---|
| Distance | 31 | 4,781.20 | 3,648.00 | 5,442.72 | 158 |
| Time | 32 | 5,033.33 | 3,370.50 | 4,662.60 | 204 |
| Mixed | 32 | 5,033.30 | 3,370.50 | 52,303.72 | 201 |

Time mode accepts approximately 252 additional metres to reduce adjusted time by 277.5 seconds (7.6 percent). This is direct evidence that congestion changes the route. Mixed cost has a different unit, so only values within mixed mode are comparable.

CHAPTER 8

# Multi-Location Optimization

## 8.1 Problem Definition

Given a depot and several required stops, the system must choose an order, find the best road path between each consecutive pair, and return to the depot. The pairwise cost matrix is intended to use A* with the selected evaluator. Nearest Neighbor gives a fast approximate order; Held-Karp searches all subset states and guarantees the best tour over the selected locations.

## 8.2 Experimental Comparison

The depot is vertex 125 and the supplied visit order is $65 \rightarrow 192 \rightarrow 155 \rightarrow 81$. Mixed mode is used. Because the current multi-location module passes an obsolete `heuristic_type` keyword, the experiment used a compatibility wrapper that discards this keyword and calls the current A* implementation. No route costs were otherwise changed.

Table 8.1: Original and optimized multi-location orders

| Method | Visiting order (including return) | Path nodes | Mixed cost |
|---|---|---|---|
| Original input | 125 - 65 - 192 - 155 - 81 - 125 | 61 | 126,552.260 |
| Nearest Neighbor | 125 - 65 - 155 - 192 - 81 - 125 | 70 | 151,739.768 |
| Held-Karp DP | 125 - 192 - 155 - 81 - 65 - 125 | 52 | 112,227.134 |

Held-Karp improves the original order by 11.32 percent and Nearest Neighbor by 26.04 percent. Nearest Neighbor is worse than even the original order in this case, illustrating that locally cheapest decisions do not guarantee a good cycle. Held-Karp is optimal for the five selected locations under the computed pairwise mixed costs, but its exponential growth limits it to small stop sets.

## 8.3 Integration Status

The algorithms and reconstruction logic exist in `src/algorithms/multi_location.py`, but they are not dispatched by the API or displayed by the frontend. The obsolete keyword call must be removed, then API schemas should accept a stop list and return visiting order, detailed path, and cost before this requirement is considered end-to-end complete.

CHAPTER 9

# Program Instructions

## 9.1 Installation and Setup

Prerequisites are Python 3.10 or later and Node.js 18 or later. From the project root, install and run the backend:

Listing 9.1: Backend setup

```
python -m pip install -r requirements.txt
python -m uvicorn backend.api:app --reload --port 8000
```

In a second terminal, run the frontend:

Listing 9.2: Frontend setup

```
cd frontend
npm install
npm run dev
```

Open http://localhost:5173. Before end-to-end use on the inspected branch, change the frontend request from `/search` to `/api/search` and align algorithm labels with the backend map. These are integration fixes, not extra runtime dependencies.

## 9.2 GUI Usage

1. Select a start and destination by dropdown or map interaction.
2. Optionally add intermediate stops.
3. Choose Default, Time, Distance, or Cost optimization.
4. Choose DFS, BFS, Dijkstra, or A* from the current GUI list.
5. Press **Search**; inspect the path and route explanation panel.
6. Use the playback buttons to move through steps, then press Reset for a new query.

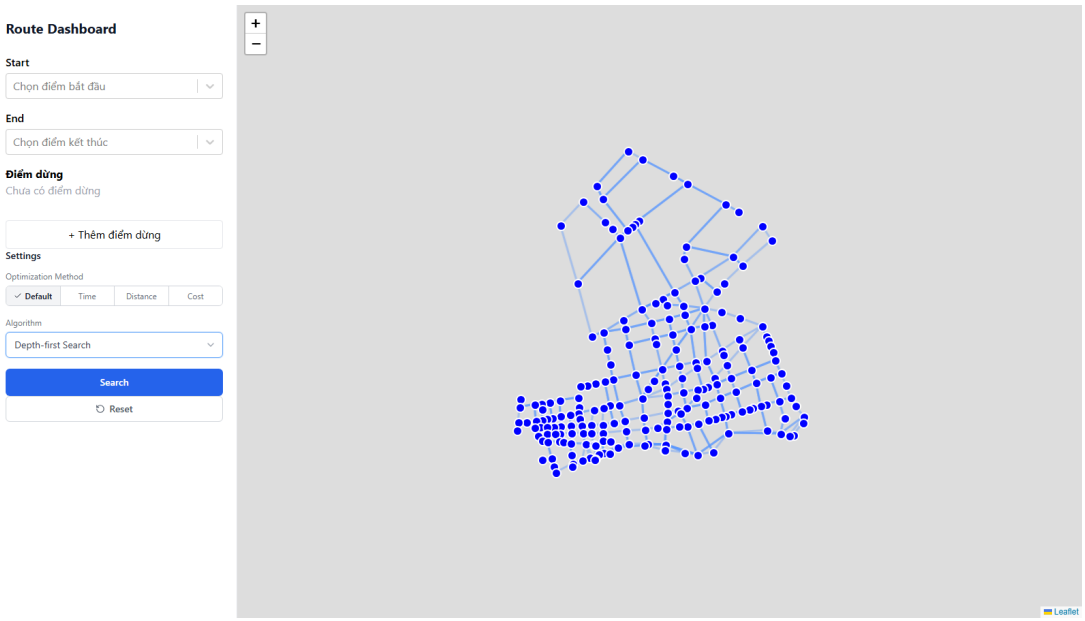


Figure 9.1: Current React interface displaying the 212-node road graph

## 9.3 Example Input and Output

A direct backend request uses the schema below:

Listing 9.3: Example search input

```
POST /api/search
{
  "start": "123",
  "end": "151",
  "algorithm": "astar",
  "optimization": "distance"
}
```

The response contains `algorithm_name`, `path`, `explored_nodes`, `total_cost`, `total_distance`, `total_time`, `explored_count`, `execution_time_ms`, names, and coordinates. For this example, A* returns 31 path vertices, 4,781.20 metres, cost 5,442.72, and 158 expanded vertices in the recorded run.

## 9.4 Route Explanation Example

The distance-mode A* route is selected because it has the lowest effective distance cost among the tested weighted methods. BFS uses fewer road segments but its cost is 8.0 percent higher. The time-mode alternative is physically longer, yet its adjusted time is 277.5 seconds lower because it avoids more costly traffic conditions. A* guarantees optimality only when its heuristic is admissible/consistent; Greedy Best-First provides no such guarantee.

CHAPTER 10

# Limitations and Future Work

## 10.1 Development Challenges

The main challenges were transforming road data into a directed graph, maintaining comparable metrics across algorithms, choosing traffic penalties, and coordinating evolving backend/frontend interfaces. Directed connectivity also makes a naive endpoint selection unreliable. Multi-location optimization adds a second level of search because every pair of required stops needs a road path before the visiting order can be optimized.

## 10.2 Current Limitations

- Traffic and risks are static simulated values rather than live observations.
- The graph covers a limited urban area and contains vertices outside the largest strongly connected component.
- Risk penalties and mixed-mode conversion are hand-designed, not statistically calibrated.
- Reported route time is aggregated with a congestion formula that is not identical to every evaluator mode.
- Frontend and backend route paths/algorithm labels are misaligned on the inspected branch.
- The frontend overwrites the returned path with a temporary direct waypoint list.
- Playback controls are not yet linked to real frontier/exploration states.
- Route explanation text is generic and does not identify congested edges or compare alternatives dynamically.
- Multi-location methods are not exposed through the API and contain one obsolete function argument.
- Automated tests do not yet cover API contracts, optimality, disconnected graphs, or repeated benchmarks.

## 10.3 Future Work

The highest-priority work is to align the API path and algorithm naming, remove the temporary frontend path override, synchronize the multi-location call, and add contract tests. The

exploration list should drive the visualizer, while the explanation generator should cite actual high-congestion and risk edges.

Longer-term extensions include real-time traffic feeds, map API integration, time-dependent edge weights, alternative-route generation, turn restrictions, bidirectional A*, support for multiple vehicles, and calibrated cost weights. Experiments should repeat many reachable source-target pairs and report mean, median, and variance instead of relying on one sub-millisecond run.

## 10.4 Conclusion

The repository provides a substantial classical-search foundation: a realistic Vietnamese traffic graph, six single-route algorithms, traffic-aware objectives, a REST service, and an interactive map. Reproducible results show the expected optimality/efficiency trade-offs and demonstrate the value of optimizing beyond distance. Held-Karp also provides a strong small-scale multi-location result. However, the documented integration gaps must be closed before the GUI, step-by-step explanation, and multi-location requirements can be considered fully complete.

# Bibliography


[1] Introduction to Artificial Intelligence, *Lab 1: Search Algorithms for Vietnamese Traffic Route Optimization*, course specification, 2026.

[2] Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2021.

[3] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 1968.

[4] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.

[5] University of Science, VNU-HCM, *HCMUS Brand Identity and Logo Usage Guide*, https://hcmus.edu.vn/he-thong-nhan-dien-thuong-hieu/, accessed August 15, 2026.

[6] FastAPI, *FastAPI Documentation*, https://fastapi.tiangolo.com/, accessed August 15, 2026.

[7] Meta Open Source, *React Documentation*, https://react.dev/, accessed August 15, 2026.

# Repository Structure

| | |
|---|---|
| `data/` | Road graph JSON files |
| `src/models.py` | Graph and cost evaluator |
| `src/heuristics.py` | Haversine/time heuristics |
| `src/algorithms/base.py` | Common search types |
| `src/algorithms/uninformed.py` | DFS and BFS |
| `src/algorithms/informed.py` | UCS, Dijkstra, A*, Greedy |
| `src/algorithms/multi_location.py` | Nearest Neighbor, Held-Karp |
| `backend/api.py` | FastAPI endpoints |
| `backend/schemas.py` | Request/response schemas |
| `frontend/src/` | React and Leaflet interface |

CHAPTER B

# Representative Optimal Route

For the two-location distance experiment, UCS, Dijkstra, and A* return:

123 → 172 → 171 → 137 → 211 → 47 → 105 → 200 → 210 → 98 → 208 → 112 → 42 → 40 → 41 → 80 → 65 → 66 → 99 → 146 → 25 → 72 → 74 → 78 → 45 → 75 → 58 → 101 → 181 → 182 → 151.

CHAPTER C

# Requirement Traceability

| Item | Required topic | Location in this report |
|---|---|---|
| (a) | Group information, contributions, completion | Chapter 1 |
| (b) | Problem context and usefulness | Chapter 2 |
| (c) | Graph, state, transitions, attributes, cost | Chapter 3 |
| (d) | Source, locations, values, assumptions | Chapter 4 |
| (e) | Principles, examples, heuristic, guarantees | Chapter 5 |
| (f) | Flowcharts, modules, GUI interaction | Chapter 6 |
| (g) | Theory and actual algorithm comparison | Chapter 7 |
| (h) | Multi-location method, orders, guarantee | Chapter 8 |
| (i) | Setup, GUI use, examples, screenshot | Chapter 9 |
| (j) | Challenges, limitations, extensions | Chapter 10 |